厦门大学课程考试（A 卷）

UNIX系统程序设计

学年学期：201520161　2013 年级计算机科学与技术专业

一、简答下面问题（36 分）

1. 如果两个常规文件的长度一样，那么是否意味着它们占用相同大小的磁盘空间？为什么？
2. 读/写磁盘文件时，每次使用系统调用 read 和 write 是否都意味着读写磁盘驱动器？为什么？
3. 如果有一个文件，它的权限被设置成文件的所有者不可读。那么文件的所有者是否能将其改为自己可读？如果其他用户可读，那么文件的所有者是否能将文件改为自己可读？
4. 在作业控制中，进程组和进程对话有什么用途？
5. 如果存在一个未决信号，如何在不执行相关的信号处理函数的前提下清除它？
6. 为什么要避免进程僵尸？请给出两种避免进程僵尸的方法。
7. 从 main 函数返回是否终止进程？函数 exit 与 `_exit` 有什么区别（至少指出两点）？
8. 什么是系统调用？是否有在 C 语言标准里定义的库函数是系统调用？
9. 如果删除一个已经打开的文件，那么能否继续读写这个打开的文件？
10. 子进程的 ID 是大于 0 还是等于 0？它是否与父进程共享内存空间？它继承父进程对信号的处理方式吗？
11. 什么是竞争条件（race condition）？使用信号机制是否可以避免？
12. 函数不可重入的原因是什么？

二、（15 分）

阅读教材程序清单 4-7（p99），回答问题：

1. dopath 函数返回值的有什么意义？
2. 程序遍历目录树的次序是深度优先还是广度优先？为什么？
3. 程序遍历目录树时，在每个节点执行的函数是什么？完成什么功能？
4. 在 dopath 函数中，语句（不含引号）“`ptr[-1] = 0;`”是否可以省略？
5. 在程序中是否可以使用系统调用 stat 获取目录项 i 节点的信息？为什么？

三、（15 分）

阅读下列程序代码，请说明第 13、14 行代码的目的是什么?

```
1  #include "apue.h"
2  #include <sys/wait.h>
3
4  int
5  main(void)
6  {
7      char    buf[MAXLINE];  /* from apue.h */
8      pid_t   pid;
9      int     status;
10
11     printf("%% ");  /* print prompt (printf requires %% to print %) */
12     while (fgets(buf, MAXLINE, stdin) != NULL) {
13         if (buf[strlen(buf) - 1] == '\n')
14             buf[strlen(buf) - 1] = 0;
15
16         if ((pid = fork()) < 0) {
17             err_sys("fork error");
18         } else if (pid == 0) {    /* child */
19             execlp(buf, buf, (char *)0);
20             err_ret("couldn't execute: %s", buf);
21             exit(127);
22         }
23
24         /* parent */
25         if ((pid = waitpid(pid, &status, 0)) < 0)
26             err_sys("waitpid error");
27         printf("%% ");
28     }
29     exit(0);
30 }
```

四、（10 分）

下面代码是利用 sigaction 函数实现 signal 函数的可靠信号版本。若使用这个 signal 函数，则

1. 进入信号处理函数时，是否自动阻塞相同的信号?
2. 进入信号处理函数后，是否自动恢复默认的信号处理方式?
3. 若一个阻塞的低速系统调用被信号中断，那么这个系统调用是否重启?
4. 若其子进程终止，则系统是否会向父进程发送信号（如果你认为会发送，则请说明发送什么信号)?

```
#include <signal.h>
#include "ourhdr.h"
sigfunc *signal(int signo, Sigfunc *func)
{
    struct sigaction act, oact;
    act.sa_handler = func;
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;
    if (signo == SIGALRM) {
#ifdef SA_INTERRUPT
        act.sa_flags |= SA_INTERRUPT;  /* SunOS */
#endif
    } else {
#ifdef SA_RESTART
        act.sa_flags |= SA_RESTART;  /* SVR4, 4.4BSD */
#endif
    }
    if (sigaction(signo, &act, &oact) < 0)
        return(SIG_ERR);
    return(oact.sa_handler);
}
```

五、（15 分）

如果执行下面程序正常结束，则在标准输出上变量 glob 和 var 的输出值是什么?

```
#include "apue.h"
#include <sys/wait.h>

static int glob;

int
main(void)
{
    pid_t   pid;
    int     var;

    var = 0;
    if ((pid = fork()) < 0)
        err_sys("fork error");
    else if (pid == 0) {        /* child */
        glob++;
        exit(1);
    }
    if (wait(NULL) != pid)      /* wait for child */
        err_sys("wait error");

    if ((pid = vfork()) < 0)
        err_sys("fork error");
    else if (pid == 0) {        /* child */
        glob++;
        var++;
        exit(2);
    }
    printf("glob = %d  var = $d\n", glob, var);
    exit(0);
}
```

六、（9 分）

阅读下面程序，简要回答问题:

```
#include "apue.h"
#include <sys/wait.h>

int main(void)
{
    char    buf[MAXLINE];  /* from apue.h */
    pid_t   pid;
    int     status;
    printf("%% ");  /* print prompt (printf requires %% to print %) */
    while (fgets(buf, MAXLINE, stdin) != NULL) {
        if (buf[strlen(buf) - 1] == '\n')
            buf[strlen(buf) - 1] = 0; /* replace newline with null */

        if ((pid = fork()) < 0) {
            err_sys("fork error");
        } else if (pid == 0) {    /* child */
            execlp(buf, buf, (char *)0);
            err_ret("couldn't execute: %s", buf);
            exit(127);
        }

        /* parent */
        if ((pid = waitpid(pid, &status, 0)) < 0)
            err_sys("waitpid error");
        printf("%% ");
    }
    exit(0);
}
```

1. 子进程的进程 ID 等于 0 吗?
2. 程序中的函数调用 `err_ret("couldn't execute: %s", buf)` 在 `pid == 0` 时一定被执行吗? 为什么?
3. 在程序里的 waitpid 语句起什么作用? 如果省略的话，程序的行为如何?